

Computational analysis of the computer games “Paint it Black” and “ Hierarchical Structure”

13343156

CS605 Mathematics and Theory of Computer Science,
Maynooth University

May 1, 2018

1 Introduction

In this report, two computer games from Bebras Computational Thinking Challenge, “Paint it Black” and “Structure Hierarchy” are analysed to discover each of their computational power. This report will make the assumption that the reader is familiar with the book [5]. It will also make the assumption that the reader is accustomed to the proof techniques therein. Normally computational power is used as a property of machines. However in this report computational power is used as a property of languages and by extension the two computer games analysed. The following definitions can be found in Sipser’s book [5] and in a report by Prof. Naughton [4]. Computational power will be used as a property of languages, and by extension the two Bebras games as follows:

Definition 1.1. The computational power of a language (either regular, context-free, or decidable) is the minimum computational power of all of the automata that can recognise that language.

Definition 1.2. The computational power of a computer game is the computational power of the language associated with the underlying problem that players must solve when playing the game.

2 Paint it black

This game is from the UK Bebras Computational Thinking Challenge [7]. The task involves three cards, each containing nine squares which are coloured black or white. Two cards are combined to create a new card. In Figure 1 an illustration of the game is shown. Combining card A and B gives us the resulting card C.

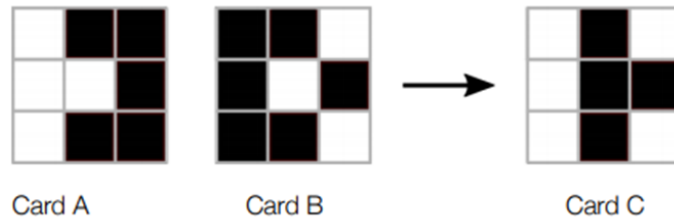


Figure 1: Combining Card A and Card B, Card C is created

The task of this game is to identify the reasoning why card A combined with card B outputs card C from Figure 2.1. The task is then to solve the output of another pair of cards, cards D and E to find how many black squares card F has, shown in Figure 2.

How many black cells will Card F have after combining Card D and Card E?

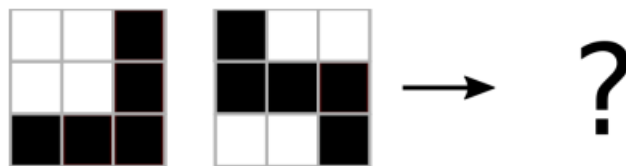


Figure 2: The given question

The answer is 3. The reasoning is that if a pair of corresponding squares from the first two cards are the same colour, the resulting square on the third will be coloured black, otherwise white as shown in Figure 3.



Figure 3: The Result

This problem relates to an XNOR Boolean logic gate. Additionally it relates to the equivalence operator which is one of the basic Boolean operations. Inter-

preting the white square as False or 0 and the Black as True or 1, the operation can be described as shown in Figure 4.

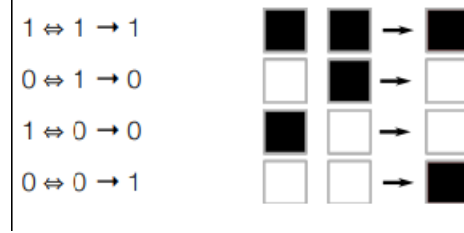


Figure 4: XNOR Operation

Before describing this game mathematically one must consider the following rules of the game. Two cards with $n \times n$ dimensions will contain n^2 number of squares and each square is coloured either ‘black’ or ‘white’. The combination of these two cards will result with another card of $n \times n$ dimensions with squares coloured black and white.

Definition 2.1. The language $CARDS_{n \times n}$ associated with the problem “Cards” is defined as $CARDS_{n \times n} = \{x \# y \# z \mid x, y, z \in \{0, 1\}^*, x_i + y_i = z_i, \forall i \in [0, n], |x| = |y| = |z| = n^2, n \in \mathbb{N}, n > 0\}$

Example 2.1. The following are examples of words in and not in the language $CARDS_{n \times n}$:

$0101 \# 1100 \# 0110 \in CARDS_{n \times n}$

$0110 \# 1110 \# 0111 \in CARDS_{n \times n}$

$0101 \# 1100 \# 1111 \notin CARDS_{n \times n}$

$010 \# 111 \# 010 \notin CARDS_{n \times n}$ (because the length of x , y or z is not equal to n^2)

$011 \# 10101 \# 1 \notin CARDS_{n \times n}$

$1000101 \# 1000101 \# 11111111 \in CARDS_{n \times n}$

2.1 One Squared Cards

To begin the analysis of $CARDS_{n \times n}$, it is best to approach computing the computational power of a simpler problem. Let’s consider “One Square Cards” such that it is identical to the original game but each card has only one square. The other rules are unchanged.

Definition 2.2. The task “One Squared Cards” is the game “Paint it Black” except each card contains a single square.

Definition 2.3. The language $CARDS_{1 \times 1}$ is defined as follows: $CARDS_{1 \times 1} = \{xyz \mid x, y, z \in \{0, 1\}, x + y = z\}$

Example 2.2. The following are examples of words in and not in the language $CARDS_{1x1}$:

$001 \in CARDS_{1x1}$
 $100 \in CARDS_{1x1}$
 $110 \notin CARDS_{1x1}$
 $000 \notin CARDS_{1x1}$

Theorem 2.1. *The language $CARDS_{1x1}$ is regular.*

Proof Idea: If the language $CARDS_{1x1}$ is regular then a finite automaton can be constructed to recognize it [1].

Proof: To prove a language is regular, a deterministic finite automaton can be constructed for the language [5]. Figure 5 is a deterministic finite automaton that recognises the language $CARDS_{1x1}$. Thus $CARDS_{1x1}$ is regular.

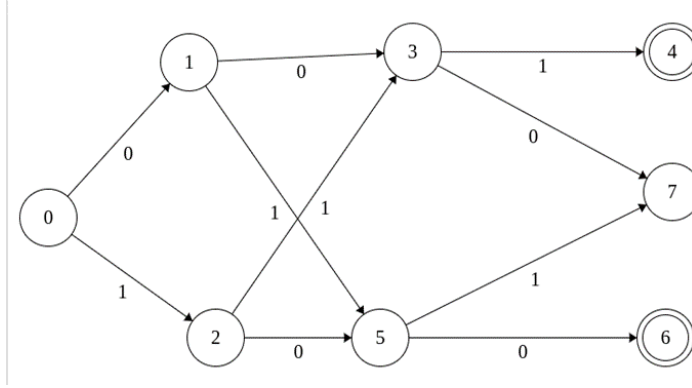


Figure 5: DFA Recognising $Cards_{1x1}$

For completeness, an alternative proof is provided below.

Proof idea. There are only 4 solutions to this problems. Namely 111, 010, 010 and 001. I can use this to prove this finite language is regular [1].

Proof (Alternative). I will prove that $CARDS_{1x1}$ is regular by showing it has a finite cardinality. The language $CARDS_{1x1} = \{111, 010, 100, 001\}$. Since $|CARDS_{1x1}| = 4$, $CARDS_{1x1}$ cardinality is finite. Since all finite languages are regular [4], $CARDS_{1x1}$ is regular.

2.2 Paint It Black with n dimensions

The language $CARDS_n = \{x\#y\#z \mid x, y, z \in \{0, 1\}^*, x_i + y_i = z_i, \forall i \in [0, n], |x| = |y| = |z| = n, n \in \mathbb{N}, n > 0\}$

Theorem 2.2. *The language $CARDS_n$ is not context-free.*

Proof Idea: We will assume $CARDS_n$ is context-free. We will then use the pumping lemma [2] to prove by contraction that this is not true.

1. For each $i \geq 0$, $w^i xy^i z$
2. $|vy| > 0$
3. $|vxy| \leq p$ where p is the pumping length

Let $s=0^p\#1^p\#0^p$.

There are two possible cases that our strings v and y can have:

1. v , y contain only one type of symbol
2. v or y contain a mixture of symbol

However case 1 fails because the third symbol type will not have the same number of occurrences. Case 2 will also fail because we will have out of order elements. S cannot be pumped which is a contraction of our assumption. Therefore $CARDS_n$ is not context-free.

Theorem 2.3. *The language $CARDS_n$ is Turing decidable.*

Proof: If a Turing machine decides a language, that language is decidable. A Turing Machine M is constructed that recognises $CARDS_n$, shown in Figure 6. According to Sipser [3], $CARDS_n$ is decidable since this Turing Machine decides the language $CARDS_n$.

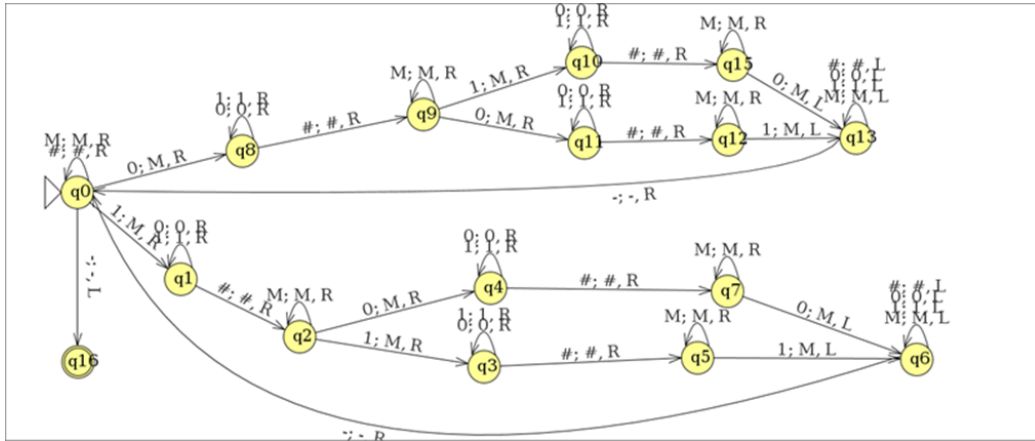


Figure 6: Turing Machine Recognising $CARDS_n$

2.3 Paint it Black with nxn dimensions

The language $CARDS_{n \times n}$ associated with the problem “Cards” is defined as $CARDS_{n \times n} = \{x \# y \# z \mid x, y, z \in \{0, 1\}^*, x_i + y_i = z_i, \forall i \in [0, n], |x| = |y| = |z| = n^2, n \in N, n > 0\}$

Theorem 2.4. $CARDS_{n \times n}$ is not context free.

Proof: Proof is identical to the proof of theorem 2.2 for the language $CARDS_n$

Theorem 2.5. $CARDS_{n \times n}$ is decidable.

Proof: A Turing Machine M will be constructed to decide $CARDS_{n \times n}$.

$M =$ “On input $\langle w \rangle$:

1. Ensure the format of the word contains three strings, each string separated with a # symbol, each of length containing only 1's and 0's. If the format is incorrect, reject.
2. Starting at the left most non marked digit of the left most string and mark it. Read the corresponding digit in the second string and mark it. Read the corresponding string in the third string.
 - (a) If the first and second digit are the same, the third digit is 1 else reject.
 - (b) If the first and second digit are different, the third digit is 0 else reject.

According to Sipser [3], $CARDS_{n \times n}$ is decidable since M decides $CARDS_{n \times n}$.

3 Hierarchical Structure

The following game was selected from Canada's 2011 Bebras problem sheet. In this game, participants must construct the encoding for given hierarchical structured folders. Encoding a folder and the folders contained within use the following rule: "each folder is encoded by its name followed by (followed by the encoding of any folders contained within it, followed by)" [6]. The Example folder is encoded as illustrated in Figure 7 : Example (X (Z ()) Q () Y ()). Folders with no folders contained within them are denoted as ().

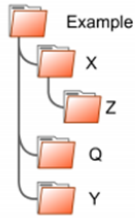


Figure 7: Hierarchical Structure Example

The problem was to match the following diagram to one of the given encodings.

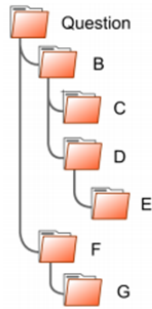


Figure 8: Hierarchical Structure Question

Possible Answers:

Question (B (C () D (E ())) F (G ()))

Question (B (C D (E)) F (G))

Question (B (C (D (E ()))) F (G ()))

Question (B (C (D () E ())) F (G ()))

The answer is Question (B (C () D (E ())) F (G ()))

In computer science this problem relates to many areas such as hierarchy structures such as trees. It relates parenthesis in mathematics so that some elements match up correctly and ensuring mathematical equations make mathematical sense with matched parenthesis. It also relates to syntax in programming languages such as Scheme.

Definition 3.1. The language *FILE* associated with the problem “Structure Hierarchy”. The name of files are restricted to ‘a’ for simplicity. Let w be the string of file names, let s be the level of hierarchy in the structure and let x be the result encoding. *FILE* is defined as $FILE = \{w\#s\#x, w=\{a\}^*, s = \{s_0 \dots s_n\} \forall i \in [0, n] s_i \in N \text{ with restriction that:}$

- 1) $s_0 = 0$
- 2) $s_{i-1} = s_i - 1$ or $s_{i-1} \geq s_i$
(ie. for any number s_i in the sequence s , s_i is either greater than s_{i+1} , or it is less than s_{i+1} by one)
- 3) x is the encoding string where each folder is encoded by its name followed by ‘(’ followed by any folders contained within it, followed by ‘)’.

Example 3.1. The following are words in and not in the language *FILE*:

$a\#0\#a() \in FILE$
 $aaa\#012\#a(a(a())) \in FILE$
 $aaa\#04\#a(((a())a())) \notin FILE$
 $a\#0\#a \notin FILE$
 $\#\#\# \in FILE$

3.1 Single Hierarchy Level

Before beginning the analysis of *FILE*, special cases of the language can be analysed to find their computational power. Considering “Single Hierarchy Level” be a special case of the general game but for each s_i in s is equal to zero.

Definition 3.2. The language $FILE_0$ is a modified version of *FILE*. It is defined as $FILE_0 = \{w\#s\#x, w=\{a\}^*, s = \{0\}^*, 3|w| = 3|s| = |x|\}$

Example 3.2. The following are words in and not in $FILE_0$:

$a\#0\#a() \in FILE_0$
 $aaa\#000\#a()a()a() \in FILE_0$
 $aa\#01\#a(a()) \notin FILE_0$
 $a\#00\#a()() \notin FILE_0$

Theorem 3.1. The language $FILE_0$ is not regular.

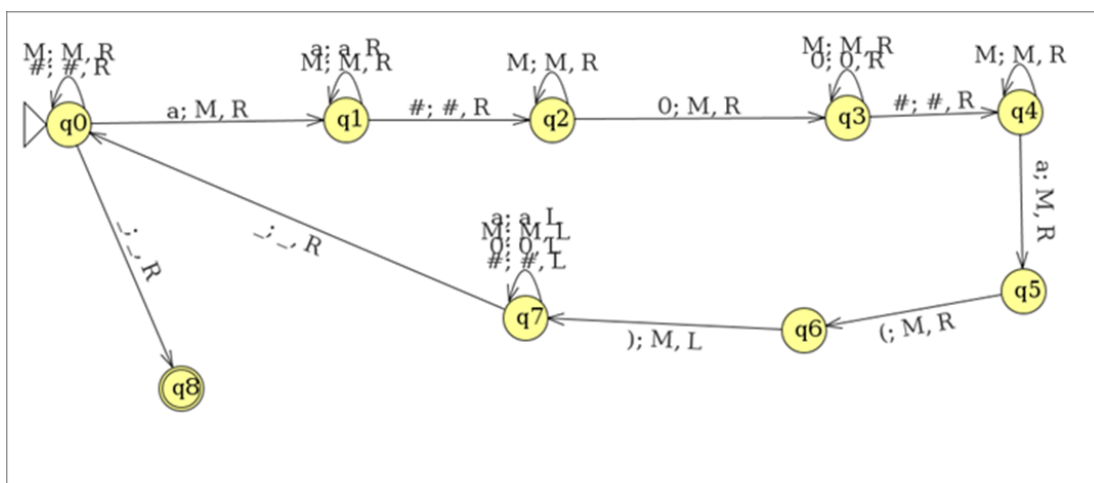
Proof idea: We will assume $FILE_0$ is regular. We will then use the pumping lemma to prove by contradiction that this is not true.

Proof: Firstly I will denote b to equal the string ‘a()’. Assuming $FILE_0$ is regular. Then each word $w \in FILE_0$ can be divided into five sub-strings, $w = uvxyz$ satisfying the three conditions mentioned in theorem 2.2. Let $w = a^p \# 0^p \# b^p$.

- 1) v and y contain only one type of symbol
- 2) v or y contain a mixture of symbols

Theorem 3.2. *The language $File_0$ is decidable.*

Proof: If a Turing machine decides a language, that language is decidable. A Turing Machine M is constructed that recognises File_0 , shown in Figure 9. According Sipser [3], File_0 is decidable since this Turing Machine decides the language File_0 .

Figure 9: Turing Machine Recognising $File_0$

3.2 File in the general form

Proofidea: We will assume *FILE* is context-free. We will then use the pumping lemma [1] to prove by contraction that this is not true.

Proof: Assuming File is context-free. Therefore each $s \in \text{File}$ can be divided into five different parts, $s=uvwxy$ which satisfies the three usual conditions stated in Theorem 2.2. Let $s=a^p \# 0^p \# a^p$.

1. v , y contain only one type of symbol
2. v or y contain a mixture of symbols

However case 1 fails because the third symbol type will not have the same number of occurrences. Case 2 will also fail because we will have out of order

elements. Therefore S cannot be pumped which is a contraction of our assumption. Hence $File$ is not context-free.

Theorem 3.4. *The language $File$ is decidable.*

Proof: A Turing Machine M will be constructed to decide $File$.

$M =$ “On input $\langle w \rangle$:

1. Ensure the format of the word is has three strings, separated by a $\#$ symbol. The first string containing only a ’s, the second string containing only integers and the third containing only ‘(’, a ’s and ‘)’.

2. If the second string is not the same length as the first string, or the third string is not three times the length of the second string, reject.

3. If each folder is encoded by its name followed by ‘(‘ followed by the encoding of any folders contained within it, followed by ‘)’ accept, else reject 4. Continue step 3 until all of string two and three are marked.

According to Sipser’s book [2], this proves that $File$ is decidable since M decides $File$.

4 Conclusion

This report has analysed the games “Paint it Black” and “Structure Hierarchy” to discover each of their computational power. By simplifying the problem “Paint it Black” so that each card only contains one square, to gradually increasing the difficulty until it can contain $n \times n$ squares, this report concludes that the “Paint it Black” problem is Turing decidable. Additionally by simplifying the problem “Structure Hierarchy” so that no folders are within folders and increasing the difficulty such that folders contain folders within itself, “Structure Hierarchy” is also Turing decidable.

References

- [1] “A language is regular iff some FA recognises it” ,Definition 1.16 in Sipser’s book [5].
- [2] “The pumping lemma for context-free languages” ,Definition 1.16 in Sipser’s book [5].
- [3] “A language is decidable if some TM decides it” ,Definition 1.16 in Sipser’s book [5].
- [4] Thomas J. Naughton. Computational analysis of various computer games. 2015.
- [5] Michael Sipser. *Introduction to the Theory of Computation*, volume 2. Thomson Course Technology Boston, 2006.
- [6] Oxford University. Canada Bebras, 2012. [Online; accessed 15-March-2018].
- [7] Oxford University. UK Bebras, 2016. [Online; accessed 25-February-2018].